



Práctica 7_f

Patrones de Diseño de
APIs

Elaborado por:
Jordan Murillo



Informe Individual — Práctica 07: Despliegue de Aplicaciones Node.js en Producción utilizando AWS, Nginx y PM2

Estudiante	Jordan Murillo
Carrera	Maestría de Software
Asignatura	Patrones de Diseño de APIs
N.º de práctica	07
Título	Despliegue de Aplicaciones Node.js en Producción utilizando AWS, Nginx y PM2
Fecha	26 de septiembre de 2026
Repositorio	Enlace

1. Objetivo

Desplegar en producción el CRUD de **Gestión de Empleados** (Express + TypeScript + MongoDB, frontend Angular) siguiendo la guía, y extenderlo a un esquema **multi-nube**:

- **Infraestructura segura en AWS EC2**, con **Nginx** como proxy inverso.
- **PM2** en modo cluster, CI/CD con **pm2 deploy + GitHub** y alertas por **Webhook (Discord)**.
- **Extensión propia**: frontend en **Azure Static Web Apps** y dominio + HTTPS unificados con **Cloudflare**.

2. Arquitectura implementada

El navegador entra por Cloudflare: `app.jmurilloai.dev` apunta a Azure y `api.jmurilloai.dev` a la EC2, donde Nginx reenvía al cluster de PM2 en `localhost:3000`, que consulta MongoDB Atlas.

Capa	Servicio	Proveedor
DNS + HTTPS	Cloudflare (proxy, modo SSL Full)	Cloudflare
Frontend	Angular en Azure Static Web Apps	Azure
API	Express en EC2 t3.small + Nginx + PM2 (cluster)	AWS
Base de datos	MongoDB Atlas	MongoDB
CI/CD frontend	GitHub Actions (build con pnpm + deploy)	GitHub / Azure
CI/CD backend	pm2 deploy production sobre SSH	GitHub / AWS
Alertas y logs	pm2-discord + pm2-logrotate	Discord

Tabla 1. Componentes de la solución.

3. Desarrollo por fases

3.1 Red en AWS

Security Group con **80 y 443** abiertos, **22** para SSH y el puerto **3000** cerrado al público. Se asoció una **IP elástica** a la instancia para que la IP no cambie al reiniciar.

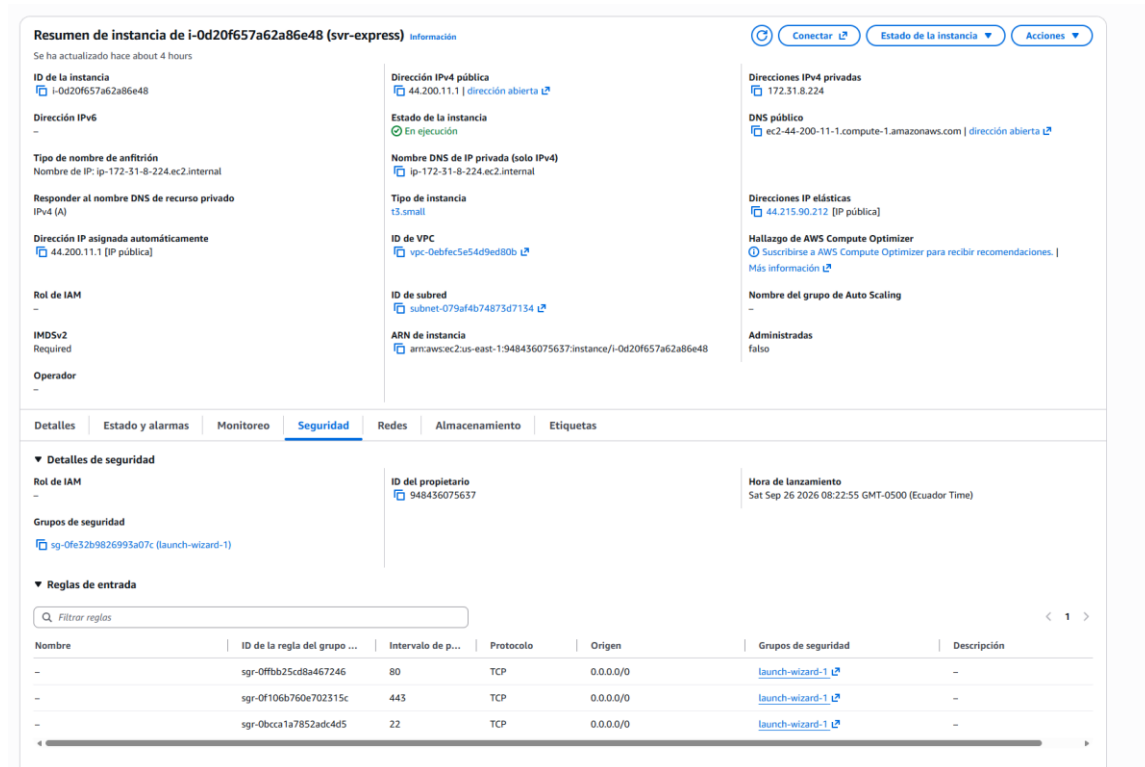


Figura 1. Instancia EC2 con IP elástica y reglas de entrada del Security Group.

3.2 Nginx como proxy inverso

Bloque server para `api.jmurilloai.dev` con `proxy_pass http://localhost:3000` y las cabeceras `X-Real-IP`, `X-Forwarded-For` y `X-Forwarded-Proto`. Se validó con `sudo nginx -t`.

```
ubuntu@ip-172-31-8-224:~$ sudo nginx -t
nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
nginx: configuration file /etc/nginx/nginx.conf test is successful
ubuntu@ip-172-31-8-224:~$
```

Figura 2. Sintaxis de Nginx validada.

3.3 Vinculación EC2 ↔ GitHub

Se generó una llave `ed25519` en la EC2 y se registró como **Deploy key de solo lectura** en el repositorio. El código se despliega en `/var/www/empleados-backend`.

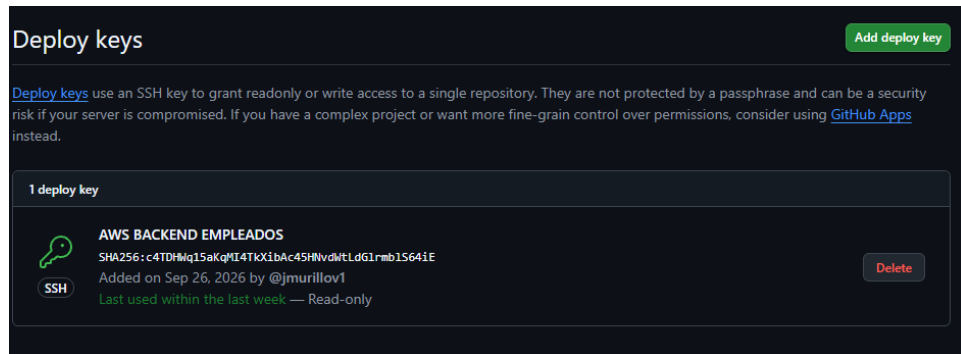


Figura 3. Deploy key registrada en GitHub.

3.4 PM2 cluster y CI/CD del backend

ecosystem.config.cjs con `exec_mode: cluster` e `instances: "max"` (2 workers en la t3.small). El post-deploy entra a `backend/` (monorepo), instala con `pnpm`, compila y hace `pm2 reload + pm2 save`. Un solo comando desde el equipo local actualiza producción:

```
jmurillov1@J0PC:~/master/module-4/practica-07/backend$ pm2 deploy production
--> Deploying to production environment
--> on host 44.215.90.212

  o deploying origin/main
  o executing pre-deploy-local
  o hook pre-deploy
  o fetching updates
  o full fetch
From github.com:jmurillov1/pda-practica-07
 94cb4e3..5dcd5ed main -> origin/main
  o resetting HEAD to origin/main
HEAD is now at 5dcd5ed feat: update CORS_ORIGIN to include Azure Static Web Apps URL
  o executing post-deploy `cd backend && pnpm install --frozen-lockfile && pnpm build && cd .. && mkdir -p logs && pm2 reload backend/ecosystem.config.cjs --env production && pm2 save`
lockfile is up to date, resolution step is skipped
Already up to date

Done in 1.5s using pnpm v10.33.0

> backend@1.0.0 build /var/www/empleados-backend/source/backend
> tsc -p tsconfig.json

[PM2][WARN] Environment [production] is not defined in process file
[PM2] Applying action reloadProcessId on app [empleados-backend](ids: [ 0, 1 ])
[PM2] [empleados-backend](0) ✓
[PM2] [empleados-backend](1) ✓
[PM2] Saving current process list...
[PM2] Successfully saved in /home/ubuntu/.pm2/dump.pm2
  o successfully deployed origin/main
--> Success
jmurillov1@J0PC:~/master/module-4/practica-07/backend$
```

Figura 4. pm2 deploy production exitoso.

```
ubuntu@ip-172-31-8-224:~$ pm2 list
```

id	name	namespace	version	mode	pid	uptime	↑	status	cpu	mem	user	watching
0	empleados-backend	default	1.0.0	cluster	63530	20m	122	online	0%	115.2mb	ubuntu	disabled
	empleados-backend	default	1.0.0	cluster	63548	20m	124	online	0%	115.6mb	ubuntu	disabled


```
Modules
```

id	module	version	pid	status	↑	cpu	mem	user
2	pm2-discord	1.0.0	67170	online	6	0%	74.1mb	ubuntu
3	pm2-logrotate	3.0.0	37456	online	0	0%	87.9mb	ubuntu


```
host metrics | cpu: 6.9% | ram usage: 68.9% | io: ↓ 0.001mb/s ↑ 0.001mb/s | ens5: ↓ 0.002mb/s ↑ 0.001mb/s | disk: ↓ 0.058mb/s ↑ 0.005mb/s / 71.8%
```

Figura 5. pm2 list: 2 workers en modo cluster online + módulos de monitorización.

3.5 Monitorización y logs

Se usó `pm2-discord` (en lugar de `pm2-notify`, que solo soporta SMTP) con el webhook del canal, activando además los eventos `error` y `exit`. `pm2-logrotate` evita que los logs llenen el disco.

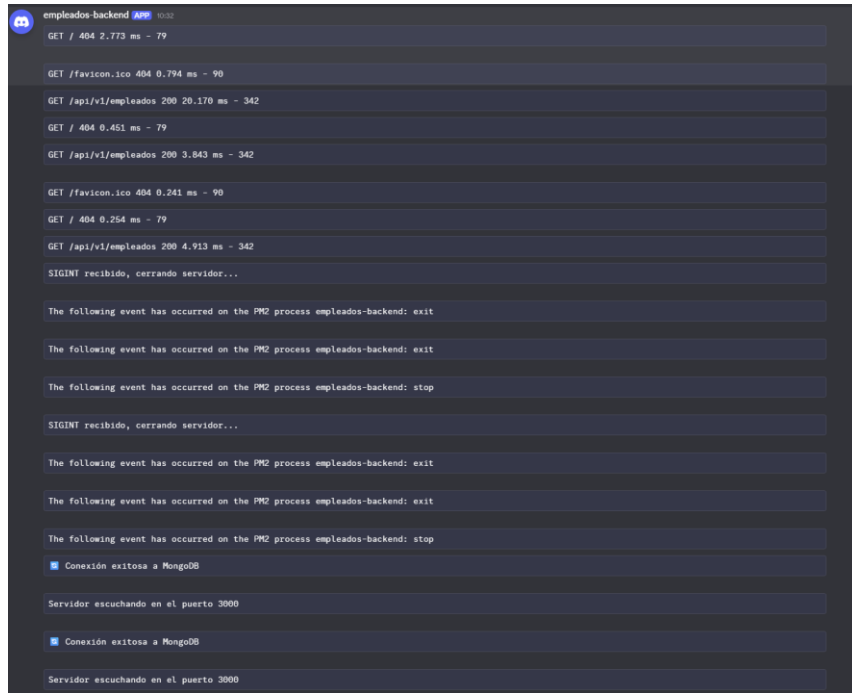


Figura 6. Logs de peticiones y eventos de PM2 llegando a Discord.

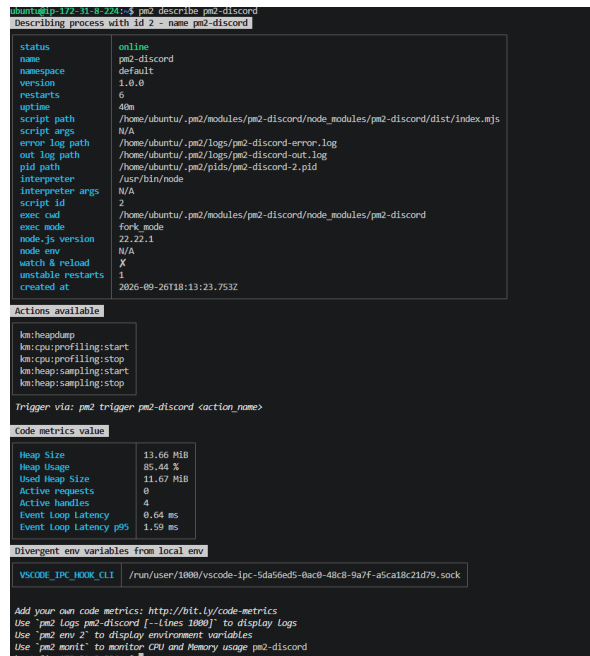


Figura 7. Módulo pm2-discord online.

3.6 Frontend en Azure Static Web Apps

El frontend Angular se publicó como Static Web App (plan Free). El workflow de GitHub Actions compila con pnpm y despliega en cada push a main.

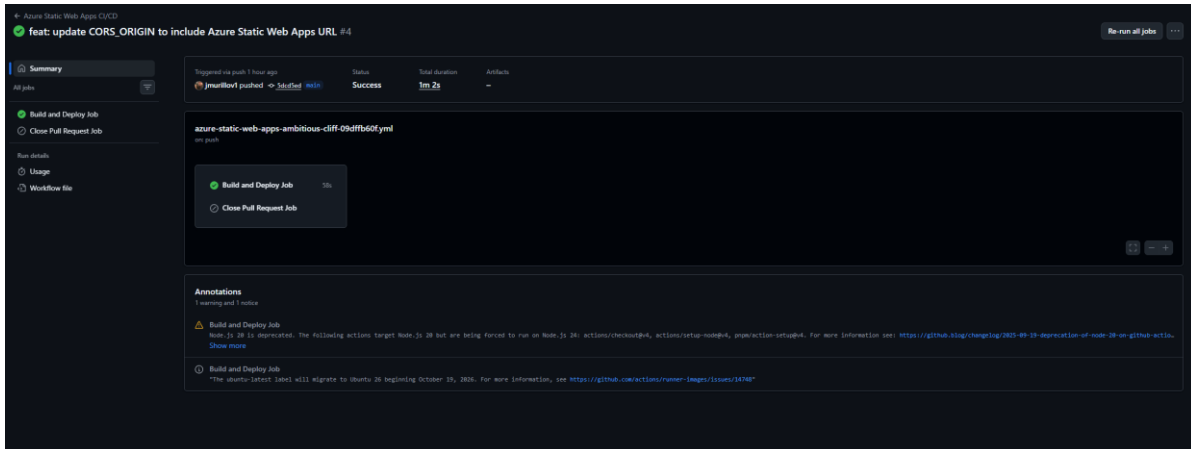


Figura 8. Workflow de GitHub Actions en verde.

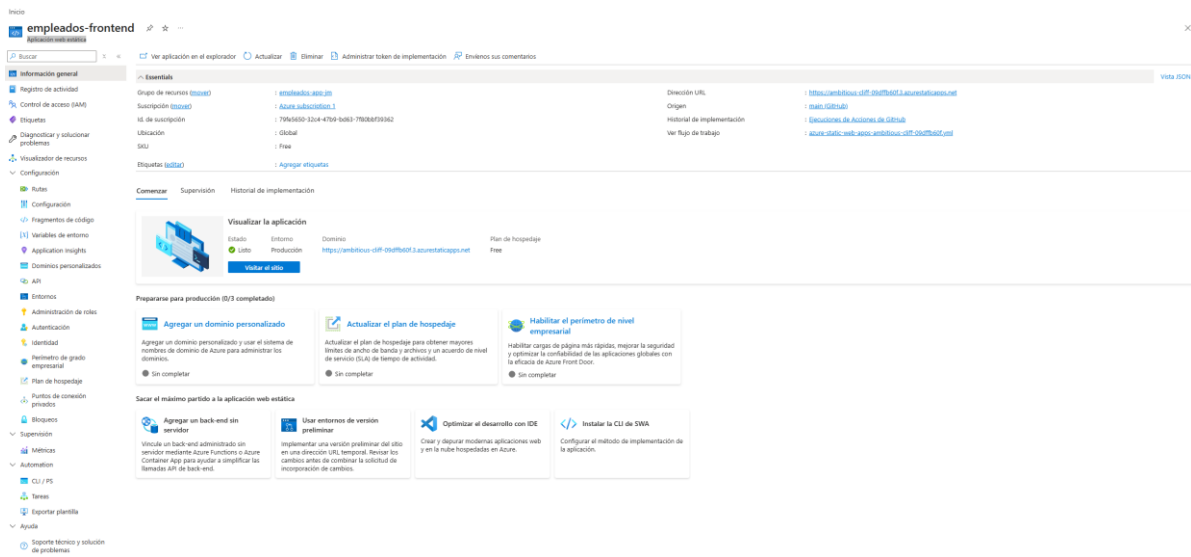


Figura 9. Static Web App empleados-frontend en Azure.

3.7 Dominio y HTTPS con Cloudflare

Registros DNS en modo **Proxied** (api como registro A hacia la IP elástica, app hacia Azure). Con el certificado de origen de Cloudflare instalado en Nginx, el modo SSL es **Full**: HTTPS de extremo a extremo.



Figura 10. Registro DNS de la API (proxied).

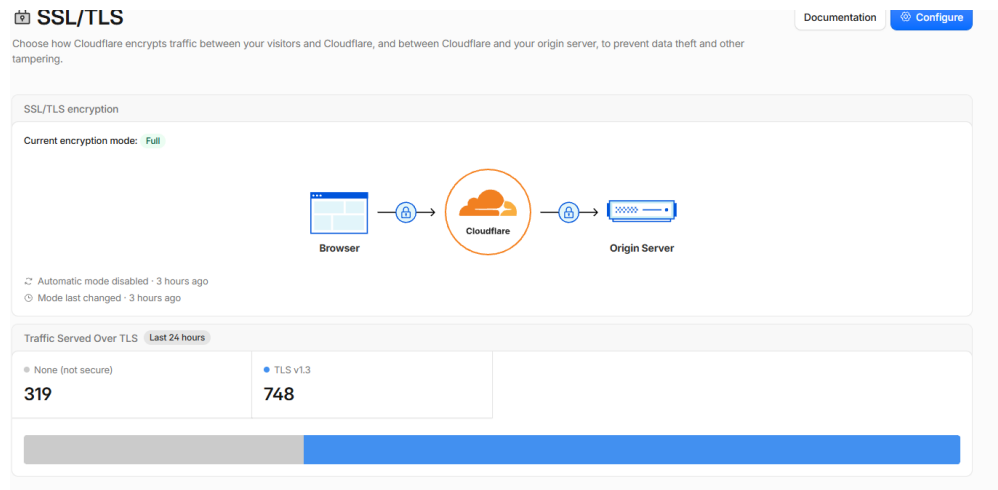


Figura 11. Cloudflare con cifrado SSL/TLS en modo Full.



Figura 12. Frontend en <https://app.jmurilloai.dev> consumiendo la API.

3.8 CORS multi-origen

Al separar dominios, el frontend pasó a usar la URL absoluta <https://api.jmurilloai.dev/api/v1> y el backend lee `CORS_ORIGIN` separado por comas (dominio de producción + `localhost:4200`).

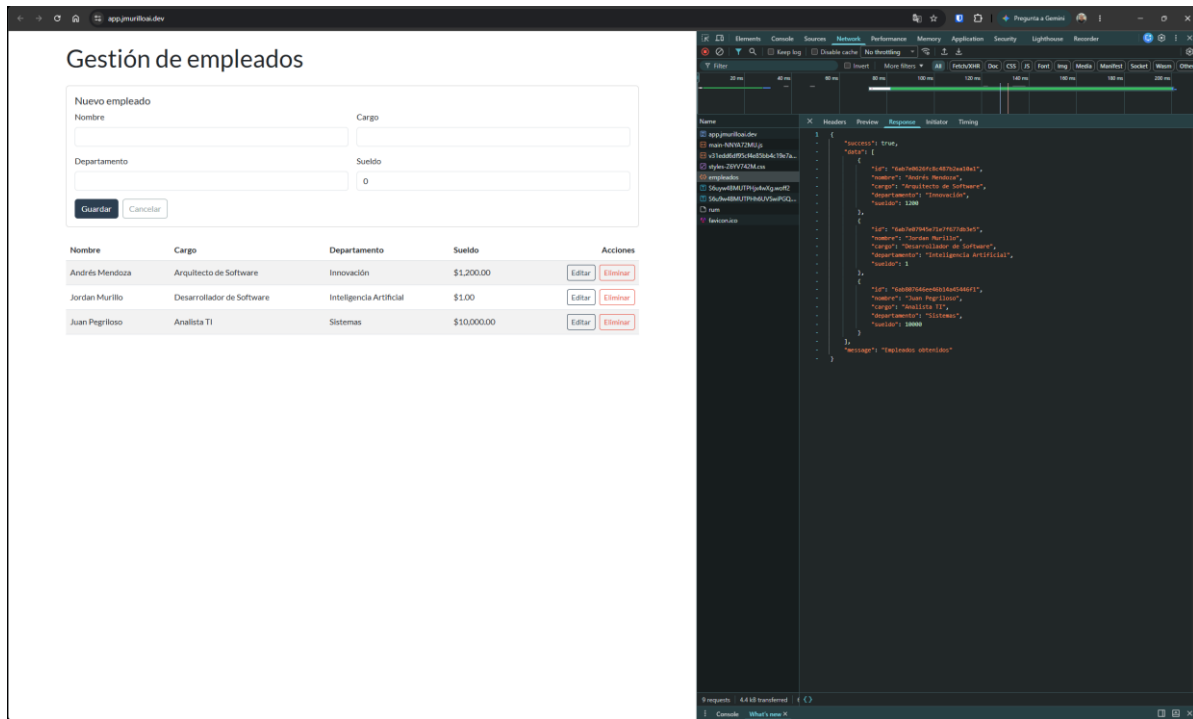


Figura 13. Petición a /api/v1/empleados desde el dominio de Azure respondiendo 200 con los datos.

4. Problemas encontrados y solución

Problema	Causa → Solución
Sin acceso SSH a la IP elástica	La IP no estaba asociada → asociarla en EC2 › IP elásticas.
`pm2 deploy` no hallaba `package.json`	Monorepo (backend/, frontend/) → el post-deploy entra a backend/.
`ecosystem.config.js` no cargaba	Proyecto ESM ("type": "module") → renombrar a .cjs.
`corepack enable` con `EACCES`	Sin permisos en /usr/bin → instalar pnpm global una vez con sudo.
No llegaban alertas	pm2-notify solo usa SMTP → pm2-discord y un pm2 set por evento.
Azure App Service sin capacidad	Región sin instancias libres → migrar a Static Web Apps.
Build de Azure fallaba (`edgesOut`)	Oryx ignora pnpm-lock.yaml → compilar en el workflow y skip_app_build: true.
Deploy a Azure: "No credentials found"	El workflow usaba otro nombre de secreto → usar el nombre exacto AZURE_STATIC_WEB_APPS_API_TOKEN_...
Frontend sin datos tras separar dominios	URL relativa /api/v1 → URL absoluta + CORS multi-origen.

Tabla 2. Problemas y soluciones.

5. Pruebas de verificación y resiliencia

Prueba	Resultado
Red	La app carga en https://app.jmurilloai.dev con HTTPS y datos (Figura 12).
Caída controlada	pm2 stop empleados-backend → alerta stop/exit en Discord al instante; se restaura con pm2 start.

Prueba	Resultado
Auto-recuperación	kill -9 a un worker → PM2 lo revive con nuevo PID (123 → 124); el otro worker sigue atendiendo.
CI/CD sin downtime	Cambio en /saludo, push y pm2 deploy: bucle de curl a /health con 240 OK y 0 fallos .
Persistencia tras reinicio	Tras sudo reboot, PM2 levanta solo la app (pm2 startup + pm2 save), sin pm2 start manual.
Degradación del frontend	Con la API detenida la interfaz sigue cargando desde Azure y solo muestra el error de la petición.

Tabla 3. Resumen de pruebas.

Caída controlada: el comando detiene los dos workers y Discord notifica cada evento.

```
ubuntu@ip-172-31-8-224:~$ pm2 stop empleados-backend
[PM2] Applying action stopProcessId on app [empleados-backend](ids: [ 0, 1 ])
[PM2] [empleados-backend](0) ✓
[PM2] [empleados-backend](1) ✓
```

id	name	namespace	version	mode	pid	uptime	♾	status	cpu	mem	user	watching
0	empleados-backend	default	1.0.0	cluster	0	0	122	stopped	0%	0b	ubuntu	disabled
1	empleados-backend	default	1.0.0	cluster	0	0	124	stopped	0%	0b	ubuntu	disabled

```
Modules
id module version pid status ⚡ cpu mem user
2 pm2-discord 1.0.0 67170 online 6 4.2% 82.0mb ubuntu
3 pm2-logrotate 3.0.0 37456 online 0 0% 88.7mb ubuntu

host metrics | cpu: 3% | ram usage: 67.8% | ens5: ↓ 0.001mb/s ↑ 0mb/s | disk: ↓ 0mb/s ↑ 0.012mb/s / 71.8%
ubuntu@ip-172-31-8-224:~$
```

Figura 14. pm2 stop empleados-backend: ambos workers detenidos.

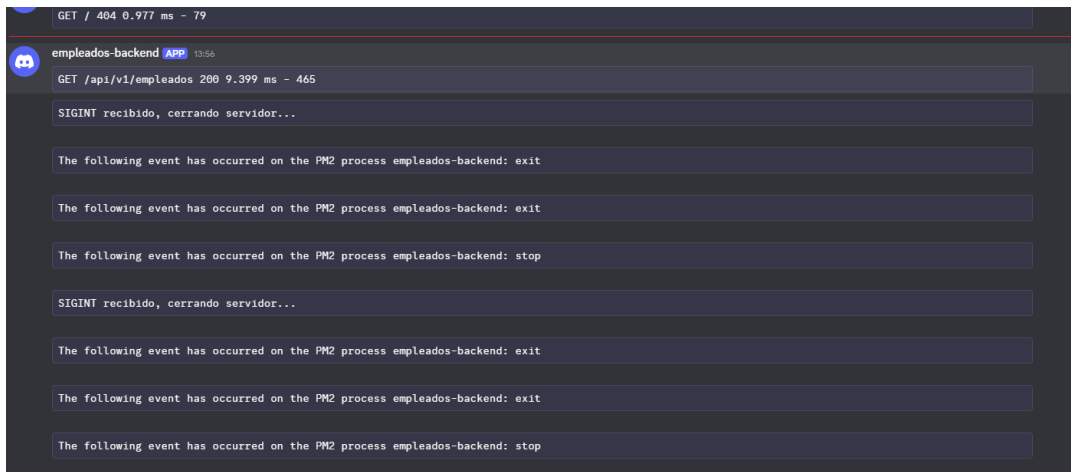


Figura 15. Alertas de Discord tras la caída.

Auto-recuperación de un worker: se mató el proceso con PID 81414; PM2 lo reemplazó al instante por el 81793 mientras el worker 1 mantuvo su uptime.

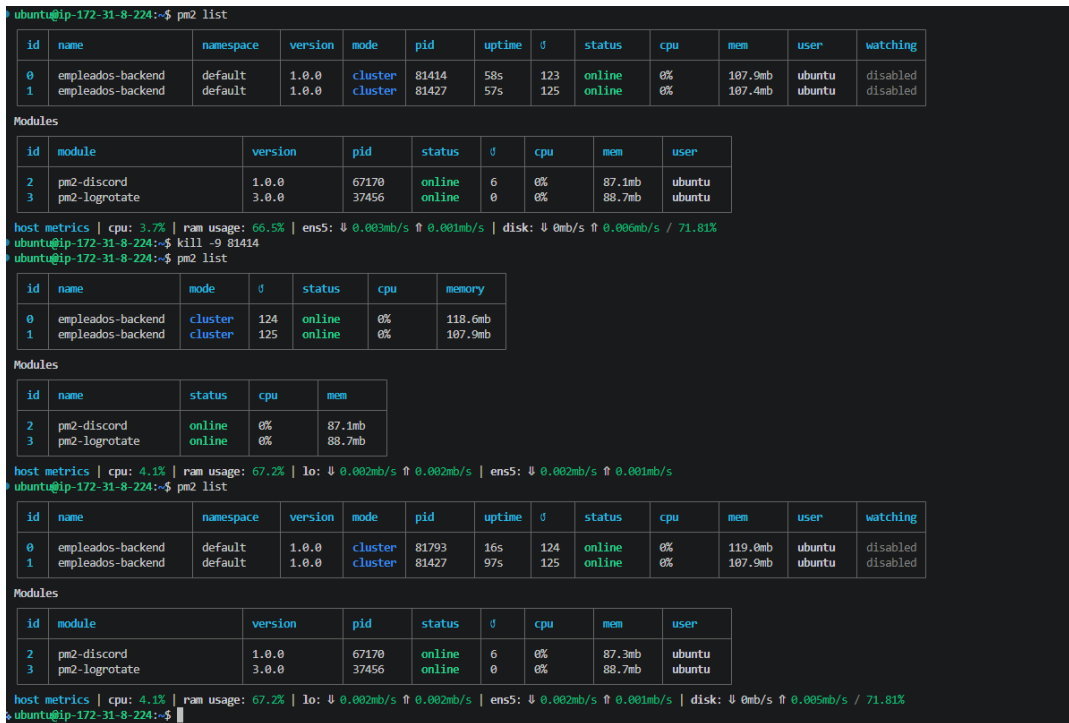


Figura 16. kill -9 a un worker y reinicio automático por PM2.

Reload sin downtime: PM2 recarga los workers uno a uno (rolling reload), así el `curl` nunca recibió un 502.

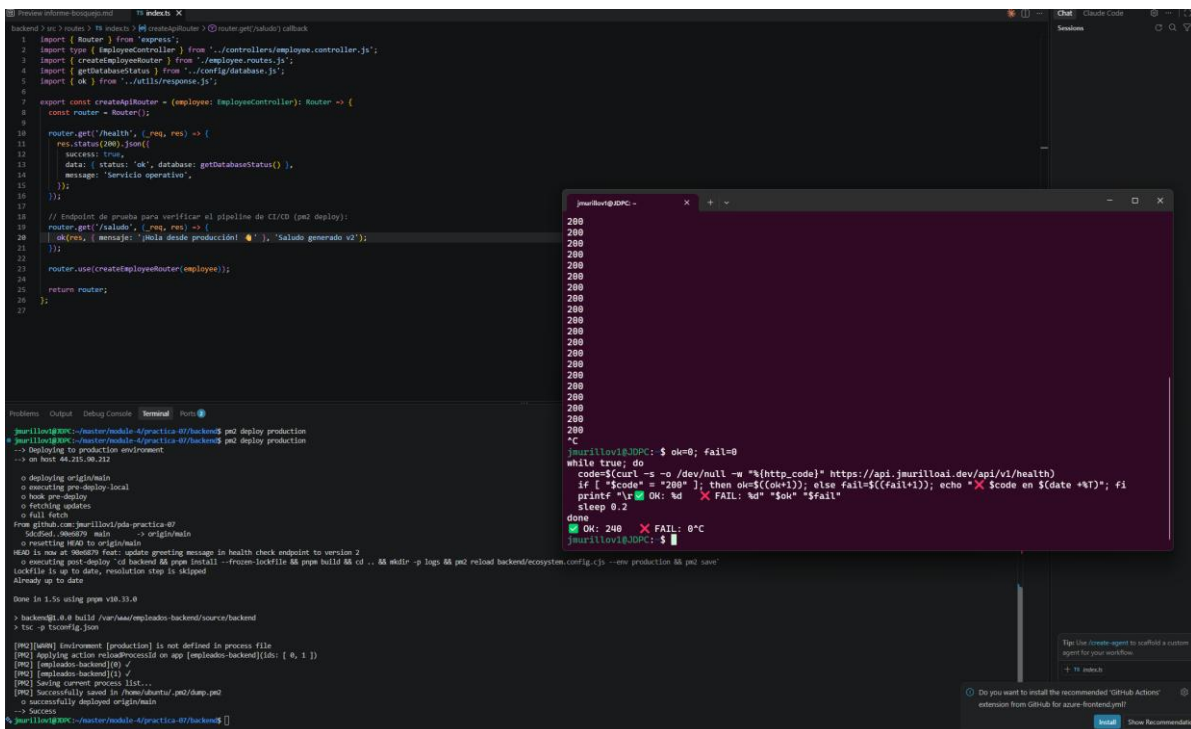


Figura 17. Despliegue del commit 90e6879 mientras el monitor registra 240 OK / 0 FAIL.

Degradación controlada: el sitio estático no depende de la EC2. El error de CORS aparece porque la respuesta 502 de Cloudflare no incluye la cabecera Access-Control-Allow-Origin.

```
ubuntu@ip-172-31-8-224:~$ pm2 stop empleados-backend
[PM2] Applying action stopProcessId on app [empleados-backend](ids: [ 0, 1 ])
[PM2] [empleados-backend](0) ✓
[PM2] [empleados-backend](1) ✓
```

id	name	namespace	version	mode	pid	uptime	↓	status	cpu	mem	user	watch
0	empleados-backend	default	1.0.0	cluster	0	0	123	stopped	0%	0b	ubuntu	disa
1	empleados-backend	default	1.0.0	cluster	0	0	125	stopped	0%	0b	ubuntu	disa

```
Modules
```

id	module	version	pid	status	↓	cpu	mem	user
2	pm2-discord	1.0.0	67170	online	6	4.1%	86.9mb	ubuntu
3	pm2-logrotate	3.0.0	37456	online	0	0%	88.7mb	ubuntu

```
host metrics | cpu: 3.1% | ram usage: 67.3% | ens5: ↓ 0.001mb/s ↑ 0mb/s | disk: ↓ 0mb/s ↑ 0.028mb/s / 71.81%
ubuntu@ip-172-31-8-224:~$
```

Figura 18. Backend detenido para la prueba.

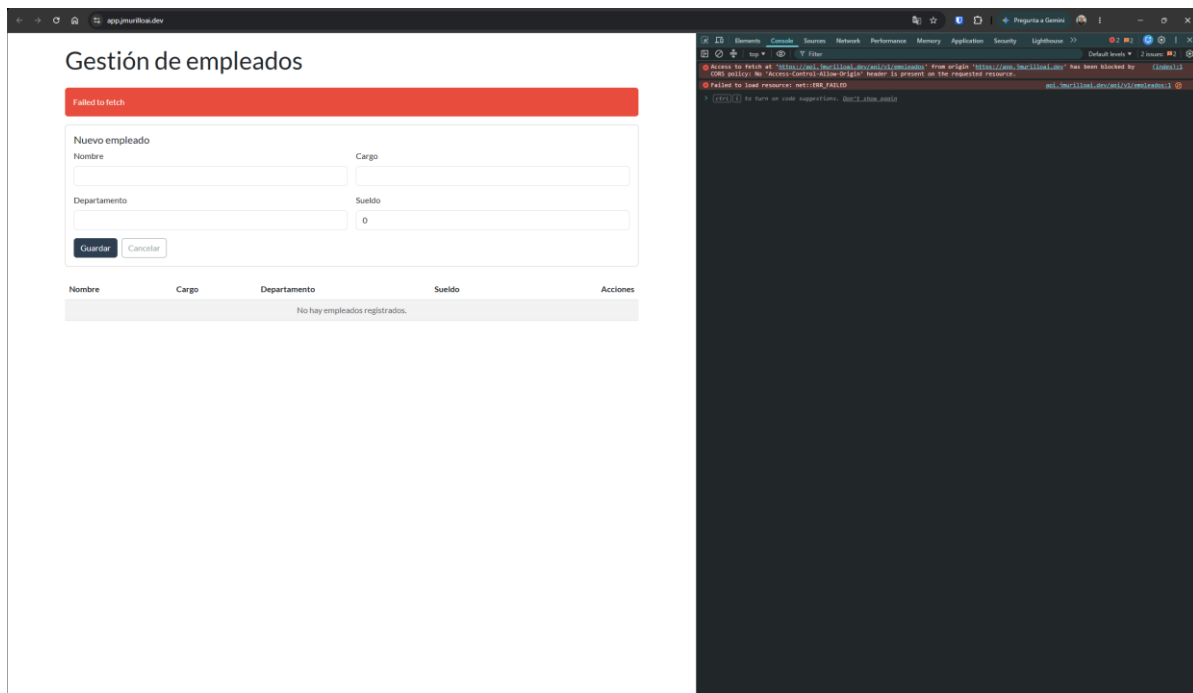


Figura 19. Frontend vivo con la API caída: muestra "Failed to fetch".

Persistencia tras reinicio: con la instancia recién arrancada (uptime: 2 min), los dos workers y los módulos aparecen online con 2 min de uptime y 0 reinicios, es decir, PM2 los levantó en el arranque.

```
ubuntu@ip-172-31-8-224:~$ uptime
19:38:37 up 2 min, 2 users, load average: 0.51, 0.55, 0.24
ubuntu@ip-172-31-8-224:~$ who -b
ubuntu@ip-172-31-8-224:~$ pm2 list
```

id	name	namespace	version	mode	pid	uptime	d	status	cpu	mem	user	watching
2	empleados-backend	default	1.0.0	cluster	1158	2m	0	online	0%	109.8mb	ubuntu	disabled
3	empleados-backend	default	1.0.0	cluster	1159	2m	0	online	0%	109.4mb	ubuntu	disabled

```
Modules
```

id	module	version	pid	status	d	cpu	mem	user
0	pm2-discord	1.0.0	1138	online	0	0%	79.9mb	ubuntu
1	pm2-logrotate	3.0.0	1148	online	0	0%	75.3mb	ubuntu

```
host metrics | cpu: 3.5% | ram usage: 64.7% | io: ↓ 0.001mb/s ↑ 0.001mb/s | ens5: ↓ 0.002mb/s ↑ 0.001mb/s | disk: ↓ 0.001mb/s ↑ 0.006mb/s / 71.82%
ubuntu@ip-172-31-8-224:~$
```

Figura 20. Estado de PM2 tras sudo reboot de la EC2.

6. Conclusiones

- **Nginx separa responsabilidades:** gestiona conexiones y cabeceras, y el backend solo ejecuta lógica de negocio.
- **El modo cluster de PM2 da disponibilidad real:** con 2 workers y rolling reload se desplegó sin un solo error (240/240); además se recupera solo de caídas de un worker y de reinicios del servidor.
- **CI/CD elimina pasos manuales:** backend con `pm2 deploy` y frontend con GitHub Actions, sin entrar a las consolas de AWS ni Azure.
- **Monitoreo proactivo:** las caídas se notifican en segundos por Discord y `pm2-logrotate` protege el disco.
- **Separar frontend y backend en nubes distintas** aísla fallos (el sitio sigue vivo sin la API), pero obliga a manejar CORS y URLs absolutas.

7. Recomendaciones

- Restringir el puerto **22** a la IP de administración (hoy está en `0.0.0.0/0`) y limitar 80/443 a los rangos de Cloudflare.
- Agregar `express-rate-limit` y `app.set('trust proxy', 1)` en el backend.
- Ejecutar `lint`, `typecheck` y `test` en CI antes de desplegar, y actualizar las actions a versiones compatibles con Node 24.

8. Referencias

- Repositorio del proyecto: <https://github.com/jmurillov1/pda-practica-07>
- Prieto, P. (2026). Guía de práctica: Despliegue de Aplicaciones Node.js en Producción utilizando AWS, Nginx y PM2. Universidad Politécnica Salesiana.
- PM2. Deployment y Cluster Mode. <https://pm2.keymetrics.io/docs>
- Microsoft. Azure Static Web Apps. <https://learn.microsoft.com/azure/static-web-apps>
- Cloudflare. SSL/TLS encryption modes. <https://developers.cloudflare.com/ssl>